

# Module 8

I<sup>2</sup>C — UVM+SV Verification

# Learning objectives

- I<sup>2</sup>C UVM agent: I2cTransaction, I2cSequence, I2cDriver, I2cMonitor, I2cScoreboard.
- Interface: i2c\_if (clk, rst\_n, start, addr, data\_in, scl, sda, clk\_div\_tick, busy, done) connects U...
- How to hook up a UVM monitor and scoreboard to a bus-oriented DUT.

# Prerequisites

- Module 7 (I<sup>2</sup>C protocol + RTL + basic TB).
- Verilator, Make, C++ compiler, UVM (UVM\_HOME or vendored UVM).

# Learning path

## Learning Path

Diagram: Learning Path

(render with mmdc when available)

flowchart LR

T1["I<sup>2</sup>C protocol recap"]

T2["Toolchain"]

T1 --> T2

T2 --> N["Next module in course"]

Extend I<sup>2</sup>C verification to **\*\*UVM+SV\*\*** — I<sup>2</sup>C agent (transaction, sequence, driver, monitor, scoreboard); run on Verilator.

# Overview

- Module 8 builds on Module 7 (I<sup>2</sup>C protocol + RTL + basic testbench):

# How to learn this module

# Suggested learning path

- Read the detailed I<sup>2</sup>C learning guide:  
I2C\_LEARNING\_GUIDE.md — what I<sup>2</sup>C is, how it works  
(START/STOP...
- Read the protocols + UVM overview:  
LEARNING\_GUIDE\_PROTOCOLS\_AND\_UVM.md — Part B § 5.  
How UVM Maps t...

# Design architecture



# RTL / block architecture

## Rtl Architecture

Diagram: Rtl Architecture

(render with mmdc when available)

flowchart TB

subgraph dut["I<sup>2</sup>C DUT (Module 7)"]

DIV["clk\_div"]

M["i2c\_master"]

DIV --> M

end

Module 8: DUT / block hierarchy (see module8/examples/).

# Verification environment architecture

## Verification Architecture

Diagram: Verification Architecture

(render with mmdc when available)

flowchart TB

SEQ["I2cSequence\naddr + data"] --> DRV["I2cDriver"]

DRV --> DUT["i2c\_master"]

DUT --> MON["I2cMonitor\nSTART + SCL/SDA sample"]

MON --> SB["I2cScoreboard\naddr and data phases"]

Module 8: UVM layers, agents, and checkers for this phase.

# 1. I<sup>2</sup>C DUT (reused from Module 7)

- i2c\_master + clk\_div; i2c\_if for UVM connection.
- Wire sequence: START → address+W → data → STOP.

## 2. UVM agent

- I2cTransaction holds `addr` + `data`; monitor fills `observed\_\*`.
- I2cMonitor reimplements baseline bus sampling inside UVM.
- I2cScoreboard checks address and data phases independently.

### 3. Course capstone

- Compare UART (async), SPI (SCLK+CS), I<sup>2</sup>C (shared bus) — same UVM skeleton, different monitors.

# **Verification & testing methods**

# Testing and closure flow

## Testing Methods

Diagram: Testing Methods

(render with mmdc when available)

flowchart TB

SEQ["I2cSequence"] --> DRV["I2cDriver"]

DRV --> DUT["i2c\_master"]

DUT --> MON["I2cMonitor"]

MON --> SB["addr + data\nscoreboard"]

Module 8: how tests, checks, and metrics close verification goals.

# 1. Two-phase scoreboard

- Separate checks for address+W byte and data byte — catches shifted or merged fields.



## 2. Reuse baseline monitor logic

- Module 7 inline monitor → Module 8 I2cMonitor for scalable tests.

### 3. Extensions

- ACK/NACK, reads, multi-master — [LEARNING\_GUIDE § 8](LEARNING\_GUIDE\_PROTOCOLS\_AND\_UVM.md#8-i2c-uvm-...

# Syllabus topics

# 1. I<sup>2</sup>C protocol recap

- START/STOP on SCL high; MSB-first address and data on SDA.

## 2. Toolchain

- ``make SIM=verilator TEST=test_i2c_uvm`;`  
  ``+UVM_TESTNAME=test_i2c_uvm`.`

# Hands-on examples

# Module 8 self-check

```
$ ./scripts/module8.sh --help
```

Terminal: module8\_check

```
[[0;36m===== [[0m
[[0;36mModule 8: Demo commands (I²C UVM example) [[0m
[[0;36m===== [[0m

From repo root:

# 1) Environment sanity:
verilator --version
make --version
echo "$UVM_HOME"

# 2) Run the I²C UVM example (inside the example directory):
cd module8/examples/i2c_uvm
make SIM=verilator TEST=test_i2c_uvm

# 3) Inspect logs and waveforms:
ls obj_dir
# If a VCD is produced:
ls *.vcd

[[1;33m[INFO] [[0m See module8/EXAMPLES.md and module8/examples/i2c_uvm/README.md for more details.
```

# Example 1: I<sup>2</sup>C UVM

- UVM phases and reporting (DRIVER, MONITOR, SCOREBOARD).
- Scoreboard checks (expected vs observed address+data from the bus).
- Final scoreboard summary (matches and mismatches).



# Demo: I<sup>2</sup>C UVM

```
$ ./scripts/module8.sh --run
```

Terminal: i2c\_uvm

```
[0:36m===== [0m
[0:36mModule 8: Run I2C UVM example (logging to /mnt/d/proj/universal-verification-methodology/lear
[0:36m===== [0m

[1:33m[INFO][0m Setting UVM_HOME to: /mnt/d/proj/universal-verification-methodology/learn_uart_spi
[Thu May 28 15:27:18 CST 2026] Running make SIM=verilator TEST=test_i2c_uvm
make: *** No rule to make target 'run', needed by 'all'. Stop.
```

# Practice & assessment

# What you should know

- Describe the I<sup>2</sup>C UVM agent and how I2cMonitor maps to SCL/SDA behavior.
- Explain two-phase scoreboard checking (address vs data).
- Run `i2c\_uvm` and interpret UVM / scoreboard output.
- Compare verification architecture across UART, SPI, and I<sup>2</sup>C modules in this course.

# Exercises

- Run i2c\_uvm
- Trace one transaction
- Change the sequence
- Compare across protocols

# Assessment checklist

- Can describe the I<sup>2</sup>C UVM agent and how it maps to I<sup>2</sup>C (addr+data, monitor on SCL/SDA).
- Can run i2c\_uvm and interpret UVM output.
- Can compare UVM structure across UART, SPI, and I<sup>2</sup>C.

# Summary & next steps

- Extend I<sup>2</sup>C verification to **\*\*UVM+SV\*\*** — I<sup>2</sup>C agent  
(transaction, sequence, driver, monitor, scoreboa...
- Complete module8/CHECKLIST.md
- Review module8/EXAMPLES.md and run each lab
- Next: Next module in course